

**Universidade do Minho**

Escola de Engenharia

Licenciatura em Engenharia Informática

Mestrado Integrado em Engenharia Informática

## **Unidade Curricular de Computação Gráfica**

Ano Letivo de 2025/2026

### **Fase 2: Transformações Geométricas**

**Jorge Rafael Machado Fernandes**  
A104168

**Diogo Teixeira Fernandes**  
A104260

**Filipe Teixeira Viana**  
A104361

# CG

|                 |  |
|-----------------|--|
| Data da Receção |  |
| Responsável     |  |
| Avaliação       |  |
| Observações     |  |

## Fase 2: Transformações Geométricas

março, 2026

# Índice

|                                               |           |
|-----------------------------------------------|-----------|
| <b>1. Introdução</b>                          | <b>1</b>  |
| <b>2. Engine</b>                              | <b>2</b>  |
| 2.1. Evolução da estrutura da cena            | 2         |
| 2.1.1. Leitura e Interpretação do XML         | 2         |
| 2.1.1.1. Configuration                        | 3         |
| 2.1.1.2. Window                               | 3         |
| 2.1.1.3. Camera                               | 3         |
| 2.1.1.4. Group                                | 3         |
| 2.2. Transformações Geométricas               | 3         |
| 2.2.1. Translação                             | 4         |
| 2.2.2. Rotação                                | 4         |
| 2.2.3. Escala                                 | 5         |
| 2.3. Renderização Hierárquica dos Grupos      | 5         |
| 2.4. Exploração da Cena                       | 6         |
| 2.4.1. Movimento Orbital                      | 6         |
| 2.4.2. Zoom                                   | 6         |
| 2.4.3. Outros                                 | 6         |
| <b>3. Cena de Demonstração: Sistema Solar</b> | <b>7</b>  |
| <b>4. Testes</b>                              | <b>9</b>  |
| <b>5. Utilitários e Estruturas de Dados</b>   | <b>10</b> |
| <b>6. Conclusões e Trabalho Futuro</b>        | <b>11</b> |

## Lista de Figuras

|           |                                                                            |   |
|-----------|----------------------------------------------------------------------------|---|
| Figura 1  | Gráfico representativo da aplicação de uma translação a um ponto           | 4 |
| Figura 2  | Gráfico representativo da aplicação de uma rotação a um ponto              | 4 |
| Figura 3  | Gráfico representativo da aplicação de uma escala a um ponto               | 5 |
| Figura 4  | Vista geral do sistema solar com cintura de asteroides visível             | 7 |
| Figura 5  | Vista aproximada destacando Saturno com anel e a distribuição dos planetas | 8 |
| Figura 6  | Resultados dos testes da Fase 2                                            | 9 |
| Figura 7  | Teste 1 - Cubo com translação                                              | 9 |
| Figura 8  | Teste 2 - Cubo, cone e esfera com translações                              | 9 |
| Figura 9  | Teste 3 - Duas esferas e um cone com translações, escalas e rotações       | 9 |
| Figura 10 | Teste 4 - Cubos com translações e rotações                                 | 9 |

# 1. Introdução

Este relatório foi elaborado no âmbito da Unidade Curricular de Computação Gráfica, no ano letivo de 2025/2026, e documenta o desenvolvimento da segunda fase do trabalho prático.

Nesta fase, o objetivo principal consistiu em estender o motor gráfico desenvolvido anteriormente, introduzindo suporte para cenas hierárquicas organizadas por grupos. Cada grupo pode conter transformações geométricas, modelos e subgrupos, permitindo a construção de cenas mais complexas e estruturadas.

A funcionalidade implementada foi validada através de vários testes de transformações e da construção de uma cena estática do sistema solar, que demonstra a aplicação hierárquica de translações, rotações e escalas.

## 2. Engine

O Engine desenvolvido na primeira fase permitia carregar e renderizar modelos tridimensionais a partir de ficheiros XML, assumindo uma estrutura simples composta essencialmente por uma lista de modelos independentes.

Na segunda fase, esta abordagem foi expandida para suportar uma organização hierárquica da cena. Em vez de existir apenas uma lista plana de modelos, passou a existir uma árvore de grupos, na qual cada grupo pode possuir transformações próprias, modelos associados e subgrupos.

Para além das configurações definidas inicialmente na execução do Engine, a aplicação permite atualizar a posição da câmara, bem como aplicar zoom. É também possível alterar o estilo de visualização da primitiva entre linhas, pontos ou preenchido, além de ativar ou desativar a visualização dos eixos da cena, proporcionando maior flexibilidade na exploração do ambiente 3D.

### 2.1. Evolução da estrutura da cena

A principal alteração introduzida nesta fase foi a substituição da estrutura plana de modelos por uma estrutura hierárquica baseada em **grupos**.

Cada grupo representa um elemento da cena e pode conter:

- Transformações geométricas;
- Modelos;
- Subgrupos.

Desta forma, as transformações aplicadas a um grupo afetam automaticamente todos os modelos e subgrupos contidos nesse grupo, permitindo uma representação modular e reutilizável de cenas complexas.

#### 2.1.1. Leitura e Interpretação do XML

A configuração do motor gráfico continua a ser carregada a partir de um ficheiro XML. No entanto, nesta fase, o processo de parsing foi estendido para interpretar uma estrutura hierárquica baseada em grupos.

O elemento XML **world** contém a configuração global da aplicação, incluindo a janela, a câmara e os grupos da cena. Cada elemento **group** pode, por sua vez, conter um bloco **transform**, um bloco **models** e outros grupos aninhados.

A leitura dos grupos é realizada recursivamente, permitindo construir em memória a árvore completa da cena. Para além disso, foram introduzidas verificações adicionais durante o parsing, nomeadamente a validação da existência de elementos obrigatórios, a deteção de atributos em falta e a rejeição de transformações duplicadas do mesmo tipo dentro do mesmo bloco transform.

De forma a suportar corretamente cenas com vários grupos no topo do elemento world, é construído internamente um grupo raiz artificial, ao qual são associados todos os grupos de primeiro nível.

#### 2.1.1.1. Configuration

A classe **Configuration** mantém-se como a estrutura central do motor gráfico, armazenando as dimensões da janela, os parâmetros da câmara e o grupo raiz da cena.

Desta forma, toda a informação necessária para a renderização fica reunida num único objeto, simplificando a fase de inicialização e desenho.

#### 2.1.1.2. Window

O elemento **window** define as dimensões da janela de renderização através dos atributos `width` e `height`. Estes valores são lidos do XML, convertidos para inteiros e armazenados na classe **Window**.

#### 2.1.1.3. Camera

A configuração da câmara é obtida a partir do elemento `camera`, que contém os subelementos **position**, **lookAt**, **up** e **projection**.

A posição da câmara define o ponto de observação inicial. O vetor **lookAt** indica o ponto para o qual a câmara está orientada, enquanto o vetor **up** define a direção vertical da mesma. O bloco **projection** contém os parâmetros de projeção perspectiva, nomeadamente o campo de visão e os planos **nearP** e **farP**.

#### 2.1.1.4. Group

A classe **Group** representa a unidade fundamental da nova estrutura hierárquica da cena.

Cada instância desta classe armazena:

- Um conjunto de nomes de ficheiros de modelos associados ao grupo;
- Um vetor de pontos correspondente à geometria carregada desses modelos;
- Uma lista de subgrupos;
- Uma matriz de transformação acumulada de dimensão  $4 \times 4$ .

A matriz de transformação é inicializada como a matriz identidade e vai sendo atualizada à medida que são lidas transformações de **translação**, **rotação** e **escala**. Durante o parsing do XML, cada elemento **group** é convertido num objeto desta classe, sendo os seus subgrupos lidos recursivamente e armazenados como filhos do grupo atual.

## 2.2. Transformações Geométricas

Nesta fase foram introduzidas três transformações geométricas aplicáveis a cada grupo: **translação**, **rotação** e **escala**.

Estas transformações são lidas a partir do bloco **transform** do XML e compostas pela ordem em que aparecem. Internamente, cada transformação é convertida numa matriz  $4 \times 4$  e multiplicada pela matriz acumulada do grupo.

Se um grupo tiver uma sequência de transformações  $T_1, T_2, \dots, T_n$ , então a matriz final do grupo é dada por:

$$M = T_1 * T_2 * \dots * T_n$$

A ordem é relevante, uma vez que a composição de transformações não é comutativa, isto é:

$$T_1 * T_2 \neq T_2 * T_1$$

Por esse motivo, aplicar primeiro uma rotação e depois uma translação produz, em geral, um resultado diferente do inverso.

### 2.2.1. Translação

A translação permite deslocar um grupo ao longo dos eixos X, Y e Z.

Dado um ponto  $P = (x, y, z)$  e um vetor de translação  $T = (t_x, t_y, t_z)$ , o ponto resultante é:

$$P' = (x + t_x, y + t_y, z + t_z)$$

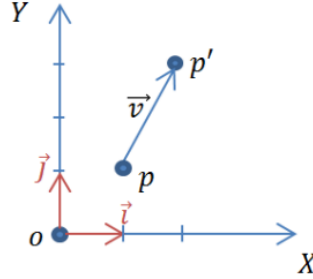


Figura 1: Gráfico representativo da aplicação de uma translação a um ponto

No ficheiro XML, esta transformação é definida como:

```
<translate x="t_x" y="t_y" z="t_z" />
```

Esta operação afeta todos os modelos e subgrupos associados ao grupo.

### 2.2.2. Rotação

A rotação permite girar um grupo em torno de um eixo arbitrário definido pelos valores  $(r_x, r_y, r_z)$  e por um ângulo  $\theta$ .

Antes de construir a matriz de rotação, o vetor eixo é normalizado. Sendo  $r = (r_x, r_y, r_z)$ , a sua norma é dada por:

$$\|r\| = \sqrt{r_x^2 + r_y^2 + r_z^2}$$

e o vetor normalizado por:

$$\hat{r} = \left( \frac{r_x}{\|r\|}, \frac{r_y}{\|r\|}, \frac{r_z}{\|r\|} \right)$$

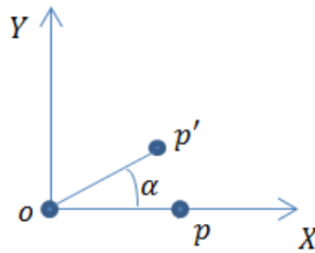


Figura 2: Gráfico representativo da aplicação de uma rotação a um ponto

No ficheiro XML, esta transformação é definida como:

```
<rotate angle="a" x="r_x" y="r_y" z="r_z" />
```

Depois da normalização do eixo, é construída a seguinte matriz de rotação  $4 \times 4$ , onde  $c = \cos(\theta)$ ,  $s = \sin(\theta)$  e  $t = 1 - \cos(\theta)$ :

$$R = \begin{pmatrix} t \cdot x^2 + c & t \cdot x \cdot y - s \cdot z & t \cdot x \cdot z + s \cdot y & 0 \\ t \cdot x \cdot y + s \cdot z & t \cdot y^2 + c & t \cdot y \cdot z - s \cdot x & 0 \\ t \cdot x \cdot z - s \cdot y & t \cdot y \cdot z + s \cdot x & t \cdot z^2 + c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



Esta matriz é multiplicada pela matriz acumulada do grupo, permitindo aplicar rotações em torno de eixos arbitrários e mantendo a coerência com a estrutura hierárquica da cena.

### 2.2.3. Escala

A escala permite alterar o tamanho de um grupo ao longo dos eixos X, Y e Z.

Dado um ponto  $P = (x, y, z)$  e fatores de escala  $S = (s_x, s_y, s_z)$ , obtém-se:

$$P' = (x * s_x, y * s_y, z * s_z)$$

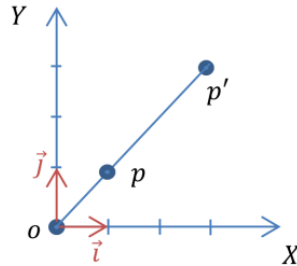


Figura 3: Gráfico representativo da aplicação de uma escala a um ponto

No XML, esta transformação é representada por:

```
<scale x="s_x" y="s_y" z="s_z" />
```

Tal como nas restantes transformações, a escala é convertida numa matriz  $4 \times 4$  e acumulada na matriz do grupo.

## 2.3. Renderização Hierárquica dos Grupos

A renderização da cena é realizada de forma recursiva, respeitando a estrutura hierárquica definida no XML.

Antes de desenhar um grupo, a sua matriz de transformação acumulada é convertida para o formato esperado pelo **OpenGL** e aplicada à matriz corrente através da função **glMultMatrixd**. Em seguida, são desenhados os pontos associados a esse grupo e, posteriormente, todos os seus subgrupos.

Para garantir que as transformações de um grupo não afetam grupos irmãos, são utilizados os mecanismos **glPushMatrix** e **glPopMatrix**. Esta abordagem assegura o isolamento de cada ramo da árvore de cena, preservando simultaneamente a herança das transformações ao longo da hierarquia.

Se um grupo filho estiver contido num grupo pai, a transformação efetiva do filho corresponde à composição da transformação do pai com a transformação local do filho, isto é:

$$M_{\text{final}} = M_{\text{pai}} * M_{\text{filho}}$$

## 2.4. Exploração da Cena

Tal como na primeira fase, foi mantida uma câmara dinâmica que permite ao utilizador explorar interativamente a cena.

A posição da câmara é calculada com base num movimento orbital em torno do ponto **lookAt**, utilizando dois ângulos,  $\alpha$  e  $\beta$ , e uma distância radial **camRadius**. As coordenadas da câmara são dadas por:

$$\text{camX} = \text{lookAt}_x + \text{camRadius} * \cos(\beta) * \sin(\alpha)$$

$$\text{camY} = \text{lookAt}_y + \text{camRadius} * \sin(\beta)$$

$$\text{camZ} = \text{lookAt}_z + \text{camRadius} * \cos(\beta) * \cos(\alpha)$$

Desta forma, o utilizador consegue observar a cena de diferentes perspetivas sem alterar a sua estrutura interna.

### 2.4.1. Movimento Orbital

O movimento orbital é controlado pelas setas do teclado, que ajustam os ângulos horizontal e vertical da câmara.

Esta abordagem permite rodar em torno da cena, facilitando a observação dos modelos a partir de diferentes pontos de vista.

### 2.4.2. Zoom

- Tecla **o**: Afasta a câmara da origem (Zoom Out).
- Tecla **i**: Aproxima a câmara da origem (Zoom In).

### 2.4.3. Outros

- Tecla **a**: Ativa ou desativa os eixos de referência.
- Tecla **r**: Reinicia a posição da câmara para a configuração inicial do XML.
- Tecla **m**: Alterna entre os modos de renderização (wireframe, pontos e preenchido).

### 3. Cena de Demonstração: Sistema Solar

Como cena de demonstração desta fase, foi construída uma representação estática do sistema solar.

O Sol foi colocado na origem da cena, enquanto os diferentes planetas foram organizados em grupos independentes com transformações próprias. Alguns planetas incluem subgrupos adicionais para representar luas, demonstrando de forma clara a herança de transformações entre grupos pai e filho.

Entre os exemplos mais relevantes desta hierarquia destacam-se:

- A Lua como subgrupo da Terra;
- Fobos e Deimos como subgrupos de Marte;
- Várias luas associadas a Júpiter;
- Um sistema próprio para Saturno, incluindo o corpo principal, anéis e satélite;
- Uma cintura de asteroides composta por cerca de 72 objetos entre Marte e Júpiter.

Esta cena permitiu validar o funcionamento da estrutura hierárquica, da composição das transformações e da renderização recursiva dos grupos.

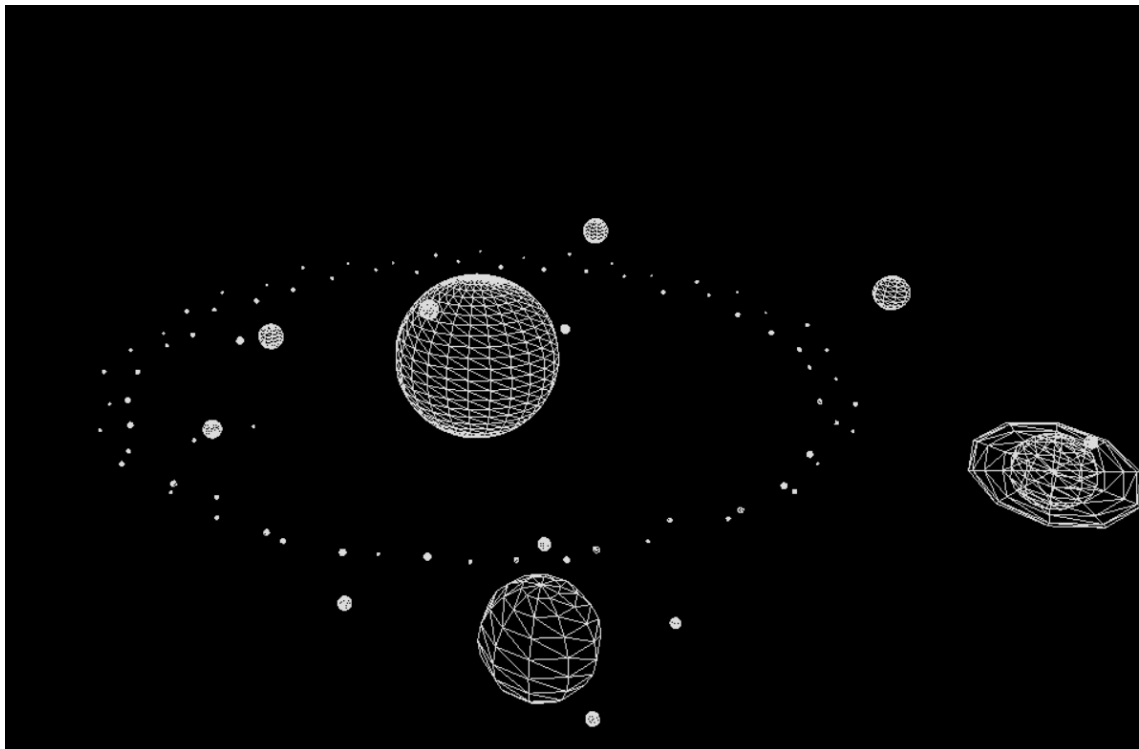


Figura 4: Vista geral do sistema solar com cintura de asteroides visível

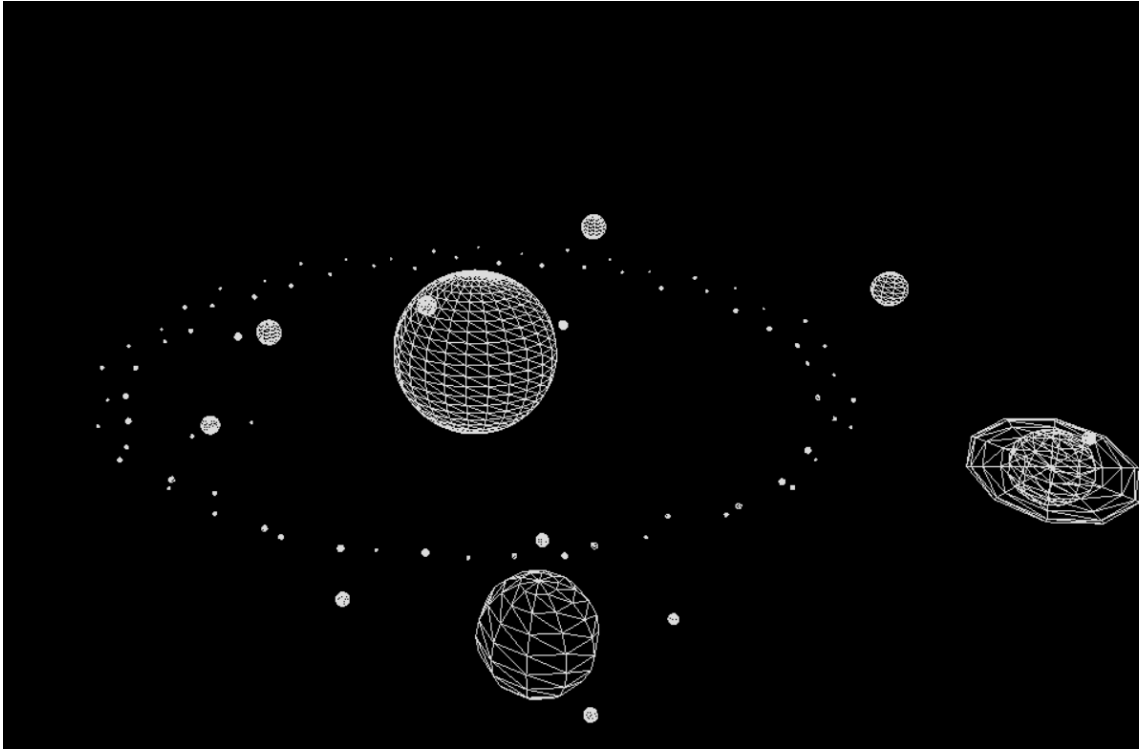


Figura 5: Vista aproximada destacando Saturno com anel e a distribuição dos planetas

A tabela seguinte resume a estrutura hierárquica da cena, indicando para cada corpo celeste o modelo utilizado, as transformações aplicadas e os eventuais subgrupos associados. As transformações são apresentadas pela ordem em que são aplicadas, dado que a sua composição não é comutativa.

| Corpo    | Modelo                            | Transformações             | Subgrupos     |
|----------|-----------------------------------|----------------------------|---------------|
| Sol      | sphere ( $r=1$ , $20 \times 20$ ) | scale 5.0                  | —             |
| Mercúrio | sphere ( $r=1$ , $10 \times 10$ ) | rotate → translate → scale | —             |
| Vénus    | sphere ( $r=1$ , $10 \times 10$ ) | rotate → translate → scale | —             |
| Terra    | sphere ( $r=1$ , $10 \times 10$ ) | rotate → translate         | Lua           |
| Marte    | sphere ( $r=1$ , $10 \times 10$ ) | rotate → translate         | Fobos, Deimos |
| Cintura  | box/sphere variados               | rotate (inclinação)        | 72 asteroides |
| Júpiter  | sphere ( $r=1$ , $10 \times 10$ ) | rotate → translate         | 4 luas        |
| Saturno  | sphere ( $r=1$ , $10 \times 10$ ) | rotate → translate         | Anel, Titã    |
| Úrano    | sphere ( $r=1$ , $10 \times 10$ ) | rotate → translate → scale | —             |
| Neptuno  | sphere ( $r=1$ , $10 \times 10$ ) | rotate → translate → scale | —             |

Tabela 1: Estrutura hierárquica do sistema solar

## 4. Testes

Nesta fase, foram realizados vários testes com o objetivo de validar o correto funcionamento das transformações geométricas e da estrutura hierárquica da cena.

Os testes incluíram:

- Aplicação isolada de translações, rotações e escalas;
- Combinação de múltiplas transformações num mesmo grupo;
- Verificação da herança de transformações entre grupos pai e filho;
- Validação de cenários previamente utilizados na fase 1, garantindo a preservação da funcionalidade já existente.

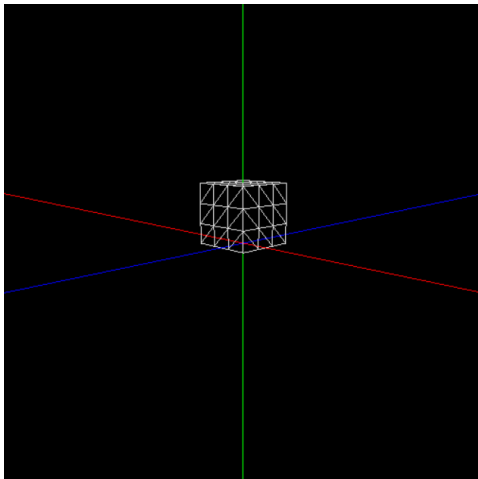


Figura 7: Teste 1 - Cubo com translação

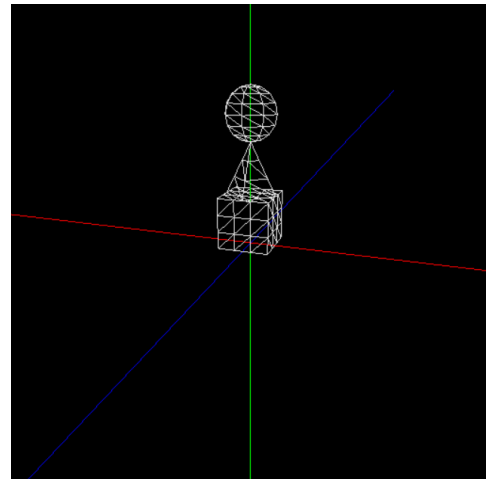


Figura 8: Teste 2 - Cubo, cone e esfera com translações

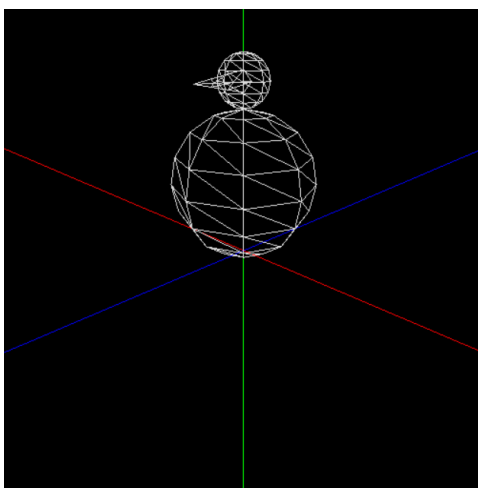


Figura 9: Teste 3 - Duas esferas e um cone com translações, escalas e rotações

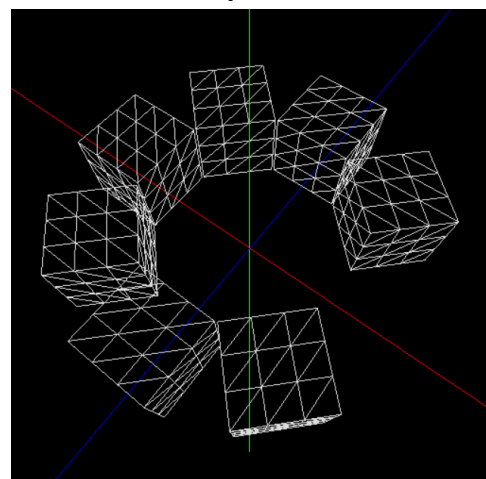


Figura 10: Teste 4 - Cubos com translações e rotações

Figura 6: Resultados dos testes da Fase 2

## 5. Utilitários e Estruturas de Dados

Para suportar a leitura e interpretação dos ficheiros de configuração, continuou a ser utilizada a biblioteca **RapidXML**. Esta biblioteca permitiu efetuar o parsing do ficheiro XML de forma simples e eficiente, sendo adequada à estrutura hierárquica introduzida nesta fase.

Ao nível das estruturas de dados partilhadas, manteve-se a utilização da classe **Point** para representar coordenadas tridimensionais. Para esta fase, a estrutura mais importante passou a ser a classe **Group**, responsável por armazenar a geometria associada a cada grupo, os subgrupos e a matriz de transformação acumulada correspondente.

A utilização desta estrutura permitiu representar a cena como uma árvore hierárquica, facilitando quer o parsing do XML, quer a renderização recursiva dos diferentes grupos.

No módulo partilhado, as funções **saveToFile** e **parse3Dfile** continuam a ser utilizadas, respetivamente, para a escrita dos modelos gerados pelo **generator** e para a leitura dos ficheiros .3d pelo **engine**.

## 6. Conclusões e Trabalho Futuro

O desenvolvimento da segunda fase permitiu expandir significativamente o motor gráfico desenvolvido anteriormente, passando de uma estrutura plana de modelos independentes para uma organização hierárquica baseada em grupos e transformações acumuladas.

A solução implementada revelou-se suficiente para representar cenas mais complexas, como o sistema solar estático, validando a leitura recursiva do XML, a composição ordenada de translações, rotações e escalas, e a herança de transformações entre grupos pai e filho.

Para além disso, foram mantidas as funcionalidades de exploração interativa da cena, incluindo movimento orbital da câmara, zoom e alternância entre diferentes modos de renderização.

Como trabalho futuro, prevê-se a continuação da evolução do projeto na fase seguinte, nomeadamente com a introdução de curvas cúbicas de Catmull-Rom para definir trajetórias animadas, rotações dependentes do tempo, a geração de modelos a partir de patches de Bézier e a otimização da renderização através de Vertex Buffer Objects (VBOs). A cena de demonstração será expandida para incluir um sistema solar dinâmico, com um cometa cuja trajetória segue uma curva de Catmull-Rom.